AIMERZ.ai
Aim.Act.Achieve

NOTES

Advanced Database Concepts

Advanced Database Concepts

Stored Procedures, Functions and Trigger in Databases

Stored procedures and functions are essential tools for efficient database management and performance optimization. They provide a way to encapsulate logic within the database, improve code reusability, security, and allow for more complex operations without constantly interacting with the database from external applications.

What are Stored Procedures and Functions?

Stored Procedure:
A stored procedure is a set of SQL statements that are saved in the database, which can be executed as a batch. Stored procedures allow you to automate repetitive tasks like updating records, processing data, or complex logic execution.

Function:
A function is a reusable block of code that performs an operation and returns a single value. Unlike stored procedures, functions are often used for calculations or data retrieval where the result is returned to the calling query.

Why Use Stored Procedures and Functions?

1. **Reusability:** Write once, use many times. Stored procedures and functions allow you to reuse code across multiple queries.
2. **Performance:** By running code directly within the database, stored procedures and functions reduce the need to send

multiple queries from the application, minimising network traffic.

3. **Security:** You can control access to the database by restricting users to execute only specific stored procedures or functions instead of giving direct access to tables.
4. **Modularity:** Break down complex tasks into manageable units with stored procedures and functions, improving code clarity and maintainability.

Stored Procedures: In-Depth

Stored procedures are generally used for operations that involve data manipulation, such as inserting, updating, or deleting records. They can perform multiple steps in a single call and support input and output parameters, making them flexible for a variety of use cases.

Example Use Case:

In an e-commerce application, a stored procedure can be used to update the stock quantities of products. Let's walk through an example that creates a stored procedure to update product stock based on the provided productID and newQuantity.

```
Unset
DELIMITER $$
CREATE PROCEDURE UpdateStock(IN productID INT, IN newQuantity INT)
BEGIN
  UPDATE Products
  SET Stock = newQuantity
  WHERE ProductID = productID;
END $$
DELIMITER ;

-- Call the procedure
CALL UpdateStock(101, 50);
```

Explanation:

1. **DELIMITER $$:**
   - SQL by default uses **;** to end statements. Since the stored procedure contains multiple SQL statements that include semicolons, **DELIMITER $$** changes the delimiter temporarily to **$$** so the database can understand that the entire block is one stored procedure.
2. **CREATE PROCEDURE**:
   - This creates a stored procedure named **UpdateStock** with two input parameters: **productID** and **newQuantity**, both of type **INT**.
3. **BEGIN and END**:
   - The **BEGIN** and **END** keywords mark the beginning and end of the stored procedure block.
4. **UPDATE Products**:
   - The **UPDATE** statement modifies the **Stock** column of the **Products** table to set the new stock level (**newQuantity**), where the product's **ProductID** matches the given **productID**.
5. **DELIMITER ;**:
   - This resets the delimiter back to the default **;** after defining the procedure.
6. **CALL UpdateStock(101, 50);**:
   - This line executes the stored procedure, updating the stock of the product with **ProductID** 101 to a new quantity of 50.

How It Works:

The **UpdateStock** procedure takes two inputs:

- **productID** identifies the product in the **Products** table.
- **newQuantity** is the new stock quantity for that product.

Whenever stock levels change (e.g., after an order or a restock), this procedure can be called to update the stock for the specified product. This allows for consistent and reusable stock updates across your system without needing to rewrite the **UPDATE** query each time. Stored procedures like this are efficient for tasks that require frequent execution and can reduce the risk of errors when modifying data manually.

Stored Functions: In-Depth

Stored Functions are typically used to return a value based on a computation or lookup. They are often employed for calculations or data transformations that need to be used in multiple queries.

Example Use Case:

In an e-commerce system, a function can be used to calculate the total value of an order. The function will take the orderID as input, compute the total order amount by multiplying the price of each item by its quantity, and then return this total value.

```
Unset
DELIMITER $$
CREATE FUNCTION CalculateTotal(orderID INT) RETURNS DECIMAL(10,2)
BEGIN
  DECLARE total DECIMAL(10,2);
  SELECT SUM(Price * Quantity) INTO total
  FROM OrderDetails
  WHERE OrderID = orderID;
  RETURN total;
```

```
END $$
DELIMITER ;

-- Call the function
SELECT CalculateTotal(102);
```

Explanation:

1. **DELIMITER $$**:
    - Similar to stored procedures, SQL uses ; to end statements. To define a function that contains multiple SQL statements, we temporarily change the delimiter to **$$** to avoid premature termination of the function definition.
2. **CREATE FUNCTION**:
    - This defines a function named **CalculateTotal** that takes one input parameter **orderID** of type **INT**. It will return a decimal value (the total order amount) formatted as **DECIMAL(10,2)**, meaning up to 10 digits total, with 2 digits after the decimal point.
3. **BEGIN and END**:
    - These keywords indicate the start and end of the function body, encapsulating the logic for the function.
4. **DECLARE total**:
    - The variable total is declared to store the result of the computation. It is also of type **DECIMAL(10,2)** to match the function's return type.
5. **SELECT SUM(Price * Quantity) INTO total**:
    - This SQL statement calculates the total order amount by multiplying the **Price** and **Quantity** of each product in the

OrderDetails table for the given **orderID**. The result of this calculation is stored in the total variable.

6. **RETURN total**:
    - The **RETURN** statement sends back the computed **total** value when the function is called.
7. **DELIMITER ;**:
    - Once the function definition is complete, the delimiter is reset to the default **;**.
8. **SELECT CalculateTotal(102);**:
    - This is an example of how the function can be used. It is called with an **orderID** of **102**, and the total order amount is returned as the result of the query.

How It Works:

This **CalculateTotal** function computes the total value of an order by multiplying the **Price** of each product by its **Quantity** for a specific **orderID**. It aggregates the results using **SUM()**, and the final total is returned as a decimal value.

By creating this function, you can quickly and easily calculate the total amount of any order just by calling the function and passing the corresponding **orderID**. The function can be reused across different queries, making it a convenient and efficient tool for order-related computations.

Triggers: Automating Actions on Data Changes

What: A trigger is a special type of stored procedure in a database that automatically executes in response to specific events, such as when data is inserted, updated, or deleted from a table. Triggers can be defined to run either **before** or **after** the specified event occurs.

Why: Triggers are useful for automating tasks based on database changes. For example, they can:

- Enforce data integrity
- Automatically update related data
- Log changes for auditing purposes
- Execute complex workflows when certain conditions are met.

By using triggers, you can ensure that certain actions are consistently executed when data is modified, reducing the need for manual intervention.

Example Use Case:

In an e-commerce system, a common use case for triggers is to automatically update the stock levels when a new order is placed. For instance, when a customer orders a product, the quantity of that product in stock should be decreased to reflect the sale.

```
Unset
DELIMITER $$
CREATE TRIGGER UpdateStockOnOrder AFTER INSERT ON OrderDetails
FOR EACH ROW
BEGIN
  UPDATE Products
  SET Stock = Stock - NEW.Quantity
  WHERE ProductID = NEW.ProductID;
END $$
DELIMITER ;
```

Explanation:

1. **DELIMITER $$**:
    - We temporarily change the SQL statement delimiter to **$$** because the trigger contains multiple SQL statements.

This prevents the function from being prematurely terminated by the default **;**.

2. **CREATE TRIGGER**:
    - This statement creates a new trigger named **UpdateStockOnOrder**.
    - The trigger is defined to execute **AFTER INSERT** on the **OrderDetails** table. This means that the trigger will activate **after** a new row is added to the **OrderDetails** table.
3. **FOR EACH ROW**:
    - The trigger operates on each row being inserted into the **OrderDetails** table. Every time an **INSERT** operation happens, the trigger executes its defined logic for that specific row.
4. **BEGIN and END**:
    - These keywords encapsulate the logic inside the trigger, allowing multiple SQL statements to be executed in sequence.
5. **UPDATE Products SET Stock = Stock - NEW.Quantity**:
    - This is the key operation performed by the trigger. It updates the **Stock** in the **Products** table by deducting the quantity of the product that was ordered.
    - The **NEW.Quantity** refers to the **Quantity** value of the row being inserted into the **OrderDetails** table. The **NEW** keyword in triggers refers to the values of the newly inserted or updated row.

6. **WHERE ProductID = NEW.ProductID**:
    - The trigger identifies the correct product by matching the **ProductID** of the row being inserted (**NEW.ProductID**) with the **ProductID** in the **Products** table.
7. **DELIMITER ;**:
    - Once the trigger definition is complete, the delimiter is reset to the default **;**.

How It Works:

This trigger is designed to automatically adjust the stock levels for products whenever a new order is inserted into the **OrderDetails** table.

- When a customer places an order, a new row is inserted into the **OrderDetails** table.
- The trigger **UpdateStockOnOrder** automatically fires **after** this row is inserted.
- The trigger deducts the ordered quantity (**NEW.Quantity**) from the stock of the corresponding product (**ProductID**) in the **Products** table.

This ensures that stock levels are kept up to date without needing manual intervention. If a product's quantity in the order is 5, the trigger will subtract 5 from the current stock. If multiple orders are placed, the trigger will handle each one automatically.

Parameter Handling in Stored Procedures

Stored procedures can accept three types of parameters:

1. **IN Parameters**: Used to pass data into the procedure.
2. **OUT Parameters**: Used to return data from the procedure.

3. **INOUT Parameters**: Used to both pass data in and return modified data.

This flexibility makes stored procedures dynamic, allowing for operations that can adapt based on the input values and return useful outputs.

Example Use Case:

In this scenario, we create a stored procedure that allows us to retrieve the available stock for a specific product in an e-commerce system. The product ID is passed as an input, and the stock quantity is returned as an output.

```
Unset
DELIMITER $$
CREATE PROCEDURE GetStock(IN productID INT, OUT availableStock INT)
BEGIN
  SELECT Stock INTO availableStock
  FROM Products
  WHERE ProductID = productID;
END $$
DELIMITER ;

-- Call the procedure
CALL GetStock(101, @stock);
SELECT @stock;
```

Explanation:

1. **DELIMITER $$**:
   - We temporarily change the SQL statement delimiter to **$$** because stored procedures contain multiple SQL statements. This prevents the procedure from ending prematurely due to the default **;** delimiter.
2. **CREATE PROCEDURE GetStock**:
   - This defines a new stored procedure named **GetStock**.

3. **Parameters:**
    - **IN productID INT**: This is the input parameter, which expects an integer representing the product's ID. The **IN** parameter is used to pass data into the stored procedure.
    - **OUT availableStock INT**: This is the output parameter, which will return the available stock quantity as an integer. The **OUT** parameter is used to return data from the procedure to the caller.

4. **BEGIN and END**:
    - These keywords define the block of SQL statements that will be executed when the procedure is called.
5. **SELECT Stock INTO availableStock**:
    - The **SELECT** statement retrieves the **Stock** value from the **Products** table based on the given **ProductID**.
    - The **INTO availableStock** part stores the result of the **SELECT** query in the **availableStock** variable, which is the **OUT** parameter of the procedure.
6. **WHERE ProductID = productID**:
    - This ensures that the procedure only retrieves the stock for the specific product ID passed in via the **IN** parameter **productID**.
7. **DELIMITER ;**:
    - After the stored procedure definition is complete, the SQL delimiter is reset to its default **;**.

Calling the Procedure:

1. **CALL GetStock(101, @stock);**:
    - This statement calls the **GetStock** stored procedure. It passes **101** as the product ID (**productID**), and the result (stock quantity) will be stored in the **@stock** variable.
2. **SELECT @stock;**:
    - This statement retrieves the value of the **@stock** variable, which holds the stock quantity returned by the **OUT** parameter of the procedure.

How It Works:

- The procedure **GetStock** is designed to take a product's ID as input and return the corresponding stock quantity.
- When you call the procedure with a specific **productID** (e.g., 101), the stored procedure will query the **Products** table for that product's stock level.
- The **OUT** parameter (**availableStock**) stores the result, which can then be retrieved and used by subsequent queries, such as **SELECT @stock;**.

This method centralises stock retrieval logic and allows for reuse across different parts of an e-commerce application, enhancing code organisation and reducing redundancy.

Transaction Management in Stored Procedures

Transactions are a critical feature in SQL, ensuring that a group of operations are executed as a single unit of work. If any operation within a transaction fails, all changes can be rolled back to prevent partial updates and maintain data integrity. Stored procedures that

handle transactions can ensure that important operations like stock updates and logging are either completed fully or not at all.

Example Use Case:

In this example, we create a stored procedure that performs two key actions as part of a single transaction:

1. **Update the stock** when an order is placed.
2. **Log the order** in a separate table.

The transaction ensures that both actions either succeed together or fail together.

```
Unset
DELIMITER $$
CREATE PROCEDURE PlaceOrder(IN productID INT, IN quantity INT)
BEGIN
  START TRANSACTION;

  -- Update the product stock
  UPDATE Products
  SET Stock = Stock - quantity
  WHERE ProductID = productID;

  -- Log the order in the OrderLogs table
  INSERT INTO OrderLogs(ProductID, Quantity, Action)
  VALUES (productID, quantity, 'Ordered');

  -- Commit the transaction
  COMMIT;
END $$
DELIMITER ;

-- Call the procedure
CALL PlaceOrder(101, 5);
```

Explanation:

1. **DELIMITER $$**:
    - We change the SQL statement delimiter to $$ so that we can define the stored procedure without prematurely ending the SQL block.
2. **CREATE PROCEDURE PlaceOrder**:
    - This defines a new stored procedure named **PlaceOrder**.
3. **Parameters:**
    - **IN productID INT**: The input parameter representing the product's ID.
    - **IN quantity INT**: The input parameter representing the quantity of the product to be ordered.
4. **START TRANSACTION**:
    - This begins a transaction block. All SQL statements within this block are treated as a single unit of work. If any statement fails, the changes can be rolled back.
5. **UPDATE Products SET Stock = Stock - quantity**:
    - This updates the stock of the specified product by reducing the stock by the **quantity** ordered.
    - The **WHERE ProductID = productID** ensures that only the stock for the specified product ID is updated.
6. **INSERT INTO OrderLogs**:
    - This inserts a new record into the **OrderLogs** table to log the action of placing an order. The **ProductID**, **Quantity**, and **Action** ('Ordered') are recorded.
7. **COMMIT**:
    - This commits the transaction. All the operations (the stock update and the order log) are finalised and saved to the

database. If any operation fails before this point, the transaction will not be committed, and all changes will be rolled back.

8. **DELIMITER ;**:
    - After defining the procedure, the delimiter is reset back to the default **;**.

Calling the Procedure:

1. **CALL PlaceOrder(101, 5);**:
    - This statement calls the PlaceOrder stored procedure, passing 101 as the product ID and 5 as the quantity to be ordered.
    - The procedure will update the stock for the product with ID 101 and log this order in the **OrderLogs** table.

How It Works:

- **Transaction Handling**: The transaction starts with **START TRANSACTION** and ends with **COMMIT**. If any operation (stock update or logging) fails, the entire transaction can be rolled back, ensuring that no partial changes are made to the database.
- **Atomicity**: Both the stock update and order log insertion are completed as a single atomic operation. If the system crashes after updating the stock but before logging the order, the stock update will not be saved either, ensuring consistency.
- **Data Integrity**: Using transactions in stored procedures guarantees that all operations succeed together, preserving the integrity of critical business processes, such as placing orders in an e-commerce system.

This design helps maintain clean and reliable data, preventing issues like incomplete orders or incorrect stock levels.

Combining Procedures, Functions, and Triggers

By combining stored procedures, functions, and triggers, you can create a powerful system for automating tasks, handling transactions, and ensuring data integrity. For example, triggers can automate stock adjustments, while stored procedures manage transactions for order placements, and functions calculate totals or other aggregate values.

Key Differences between Stored Procedures, Functions, and Triggers

Feature	Stored Procedure	Function	Trigger
Execution	Called explicitly by users	Called as part of SQL statements	Automatically executed by the database
Return Type	Can return multiple result sets	Returns a single value	Does not return a value
State Change	Can modify database state (DML)	Cannot modify database state	Can modify database state (DML)
Usage Context	For complex operations and business logic	For calculations and returning values	For automatic responses to data changes
Parameters	Can accept multiple parameters	Can accept parameters	Does not accept parameters

Views in Databases

Views are an essential tool in databases, allowing developers and analysts to simplify complex queries and secure access to data. They act as virtual tables based on the result of a predefined SQL query, allowing users to interact with them as if they were actual tables. Views provide flexibility, security, and ease of access when managing large and complex datasets.

What

A **view** is a virtual table in a database that is defined by a SQL query. Unlike physical tables that store data, views do not store data themselves; they dynamically display data that is retrieved from other tables in the database whenever the view is queried. This makes views an effective way to present data without duplicating it.

Why

- Simplification of Complex Queries: Views can encapsulate complex SQL queries, allowing users to interact with the data without needing to understand the underlying complexity.
- Data Abstraction: Views can hide unnecessary details from users, presenting a simplified version of the data. This is particularly useful for providing tailored data access to different users.
- Access Control: By omitting sensitive columns from a view, administrators can restrict access to certain data, enhancing security and ensuring that users only see what they need.

- Reusability: Once created, views can be reused in various queries, promoting consistency and reducing redundancy in SQL code.

Example Use Case:

In this example, we will create a view to summarise product sales. The view will show the total number of items sold and the total revenue generated for each product, providing a quick overview of sales performance.

```
Unset
CREATE VIEW ProductSalesSummary AS
SELECT P.ProductID,
       P.ProductName,
       SUM(O.Quantity) AS TotalSold,
       SUM(O.Price * O.Quantity) AS TotalRevenue
FROM Products P
JOIN OrderDetails O ON P.ProductID = O.ProductID
GROUP BY P.ProductID, P.ProductName;

-- Use the view
SELECT * FROM ProductSalesSummary;
```

How It Works:

1. **CREATE VIEW ProductSalesSummary AS**:
    - This statement creates a new view named **ProductSalesSummary**. The view will be defined by the SQL query that follows.
2. **SELECT P.ProductID, P.ProductName**:
    - The query selects the **ProductID** and **ProductName** columns from the **Products** table, which is aliased as **P**.
3. **SUM(O.Quantity) AS TotalSold**:

- This calculates the total quantity of each product sold by summing the **Quantity** from the **OrderDetails** table (aliased as **O**). The result is labelled as **TotalSold**.

4. **SUM(O.Price * O.Quantity) AS TotalRevenue**:
   - This computes the total revenue generated from the sales of each product by multiplying the **Price** of the items by the **Quantity** sold and summing the results. This result is labelled as **TotalRevenue**.
5. **FROM Products P JOIN OrderDetails O ON P.ProductID = O.ProductID**:
   - The query performs an inner join between the **Products** table and the **OrderDetails** table based on the **ProductID** to link the sales information with the corresponding product.
6. **GROUP BY P.ProductID, P.ProductName**:
   - The query groups the results by **ProductID** and **ProductName**, which allows the aggregation functions (SUM) to compute totals for each product.
7. **Using the View**:
   - After creating the view, you can query it like a regular table using **SELECT * FROM ProductSalesSummary;**. This will return the summarised sales data for each product.

Benefits of Using Views

1. **Convenience**: Users can access complex aggregated data without needing to write complex SQL queries repeatedly.

2. **Performance**: While views do not store data, they can improve performance for specific queries by encapsulating complex joins and aggregations, although performance can vary based on the database system and how the view is optimised.
3. **Data Security**: By restricting access to sensitive data, views can help protect information and ensure compliance with data protection regulations.
4. **Ease of Maintenance**: Changes to underlying tables do not require users to adjust their queries; as long as the view definition remains valid, it will continue to work.

Overall, views are powerful tools in SQL that enhance data accessibility, security, and usability while maintaining the integrity of the underlying data structure.

THANK YOU

AIMERZ.ai
Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan